

Agentic-Augmented Scrum (AAS)

A refined SDLC for teams where AI writes most of the code.

The core insight

In classic Scrum, the cost function being optimised was **human coding hours**. Story points, velocity, sprint length, standups, and burndown charts all exist because writing code was slow, variable, and the dominant bottleneck.

In 2026, that's no longer true. Code generation collapses to seconds or minutes — Scrum.org's evidence-based management work observes that “a 3-point story typically represented X amount of human effort. In 2026, an AI agent can generate the code for that same story in 4 seconds. This creates a crisis for Agile measurement” [1]. The work that remains — and the work that now dominates a feature's calendar time — is:

1. **Specification** — turning a fuzzy requirement into something an agent can implement correctly
2. **Validation** — reviewing, testing, and accepting AI-generated output
3. **Integration** — making the change actually work inside the existing system
4. **Handoff** — the time between an agent getting stuck and a human resolving it

The shift in effort distribution is well-documented. Industry estimation guidance now notes that AI “shifts effort from coding to higher-level tasks like requirements analysis, prompt engineering and validation of AI-generated outputs” [2]. Capgemini's Steve Jones argues more provocatively that “agentic SDLCs are too fast for Agile — the traditional two-week sprint cycle appears antiquated when AI can generate functional code in minutes” [3].

AAS keeps Scrum's predictability machinery (estimation, velocity, burndown, “what's next”) but rewires every metric to measure these new bottlenecks instead of coding effort.

The unit of work: the Spec Card

Replace the user story as the atomic unit. A Spec Card has six fields:

Field	Purpose
Intent	One sentence: what changes for the user.
Acceptance criteria	Executable tests the AI must pass (TDD-first).
Spec	The detailed instruction the agent will work from.
Validation budget	Estimated human review hours (the new “story points”).
Risk tier	T1 (autonomous merge), T2 (human review), T3 (architect approval).
Definition of Done	Tests pass + review complete + integrated + observed in staging.

The Spec Card is the contract between human intent and agent execution. The card is “ready” only when an agent could pick it up and implement it without asking further questions. **A poorly specified card is the new bottleneck** — equivalent to a poorly groomed backlog item in classic Scrum.

The TDD-first acceptance criteria are critical. Kent Beck’s augmented-coding work calls AI agents “unpredictable genies” that grant wishes in unexpected ways, and notes that “AI agents keep trying to delete tests to make them pass, forcing you to maintain vigilant oversight” [4]. Tests written before implementation act as guardrails the agent cannot reason its way around.

Estimation: Validation Hours (VH)

Story points measured human cognitive effort. That metric is dead. Scrum.org’s guidance on AI-augmented teams is explicit: assigning story points to AI agents is “a fundamental mistake” because agents “do not experience effort in the same way a human Developer does. Instead, tasks assigned to bots should be measured by their compute cost, API token utilization, and, most importantly, the human validation time required” [5].

AAS replaces story points with Validation Hours (VH) — the estimated human time required to:

- Refine the spec until an agent can act on it
- Review the generated PR
- Resolve handoffs when the agent gets stuck
- Verify the change in staging

A typical scale:

VH	What it means	Examples
0.5	Trivial: agent runs autonomously, glance-and-merge	Copy change, dependency bump, simple refactor
1	Small: one short review cycle	New endpoint with clear contract, isolated bug fix
2	Medium: spec needs care, one or two review iterations	New feature flag, modest schema change
4	Large: complex spec, multiple handoffs likely	New service integration, non-trivial migration
8+	Split it. Anything 8+ VH is a sign the card needs decomposition.	

VH is **not** wall-clock time — it’s the human-attention budget. A 2 VH card might complete in 3 hours of calendar time (because the agent worked on it while you slept), but it consumed 2 hours of your reviewing/specifying time.

The asymmetry that motivates this is captured well by Scrum.org: “A complex algorithm might take an AI agent three minutes to write, but it might take a human architect three hours to securely review and merge. If you plan your Sprint purely on the AI’s generation speed, you will create a massive, unmanageable bottleneck at the human review stage” [5].

Track separately, in parallel, **Agent Compute Cost** (tokens, dollars, minutes) for budgeting and capacity planning. This is the new “machine resource” line on the project ledger.

Cadence: the 3-day micro-sprint

Two-week sprints are too long when features can ship in a day. One-day cycles are too short to allow meaningful planning. The sweet spot most teams converge on is **3 working days** (sometimes called a “tick”).

Day	Activity	Duration
Day 0 PM	Spec session — refine cards to “ready,” set acceptance tests, assign VH	60-90 min
Day 1	Build (agents) + review (humans). Async standup via AI summary at noon.	full day
Day 2	Build + review continues. Mid-tick checkpoint at noon: are we on track?	full day
Day 3 AM	Final reviews, integration, demo.	half day
Day 3 PM	Retro (15 min, AI-summarised) → Spec session for next tick	60-90 min

This gives you **~80 ticks per year** instead of ~26 sprints. More data points means tighter estimation accuracy and faster correction when something is off.

Standups become async. An AI summarises overnight agent activity, blocked PRs, and current handoffs into a 5-bullet Slack post each morning. HPE’s developer guidance describes this pattern: agentic AI acts as “a smart

co-pilot that automatically analyzes sprint progress, flags blockers in real time, and pulls together retrospective insights” [6]. Humans only synchronously meet when the AI flags a blocker that needs discussion.

The five metrics that replace velocity

Velocity (“points per sprint”) is now meaningless — agents can inflate it 100× overnight without delivering more value. Scrum.org’s analysis notes that “if your team starts using Copilot or Cursor, and their Velocity jumps from 50 to 5,000 in one Sprint, have they actually delivered 100x more value? No. They have simply broken the metric” [1]. Track these instead:

1. Tick Throughput — VH completed per tick

Your “velocity” equivalent. Track a rolling 5-tick average. **This is what you use to forecast.** If your average is 24 VH/tick and a project is 200 VH of cards, you have ~8-9 ticks of work, or about 5 calendar weeks.

2. Validation Hours Predicted vs Actual (VH P/A)

For every card, log estimated VH at start and actual VH at completion. The ratio (target: 0.85-1.15) tells you whether your estimation is calibrated. If actuals consistently exceed estimates by 50%+, your specs are undercooked — the bottleneck is upstream of the agent. This is consistent with Augment Code’s research observation that “traditional development metrics fail when AI generates code because they miss prompt crafting time, pre-CI fixes, and context quality that determines sustainable velocity” [7].

3. Agent Efficiency Score (AES)

Adapted from the Scrum.org framework. For each agent-handled task:

$$\text{AES} = (\text{useful_output_accepted}) / (\text{useful_output_accepted} + \text{human_rework_time} + \text{handoff_time})$$

Score 0-100. Scrum.org’s guidance is direct on the threshold: “If an agent’s AES drops below 40, the Scrum Team should inspect this in the Retrospective. The solution might be to fire the agent for that specific workflow and revert to human crafting” [1]. Track AES per workflow type (frontend, migrations, tests, docs) so you know where AI helps and where it doesn’t.

4. Handoff Time (HT)

The time between an agent saying “I’m stuck” and a human resuming the work. Scrum.org reports observed handoff times of 5.5 hours in real teams, and recommends that “to improve Flow, Scrum Teams must design ‘Warm Handoffs’, where the agent provides a concise summary of the conflict, allowing the human to decide in minutes, not hours” [1]. Target: **under 30 minutes during working hours.** This metric exposes the real cost of context-switching tax.

5. Rework Rate & Escaped Defects

The quality guard. AI generates code fast; if rework rate (% of merged PRs requiring follow-up fixes within 7 days) climbs above ~15%, you’re shipping AI-induced technical debt. Scrum.org warns explicitly about “false

velocity” in AI-augmented teams, where increased throughput today becomes the outage tomorrow [8]. Pair with escaped-defect count from production. These are your DORA-equivalent reliability indicators.

The relevant industry data points: LinearB’s metrics framework recommends tracking PR Size, Review Depth, Time to Approve, and Rework Rate specifically for AI-generated code, because “an inflation in average PR size is an early indicator of inefficiencies” [9]. Axify’s 2026 metrics guide adds that “in professional settings, fewer than 44% of AI-generated code suggestions are accepted as-is” — making revision depth a leading indicator of how well the agent is calibrated to your codebase [10].

Forecasting: answering “when will it be done?”

Classic Scrum forecasting: `remaining_points / velocity = sprints`. AAS works the same way, but with VH:

```
calendar_ticks_remaining = remaining_VH / rolling_5_tick_throughput
calendar_weeks_remaining = calendar_ticks_remaining × 0.6    # 3-day ticks, 5-day weeks
```

Confidence bands matter more here than in classic Scrum, because individual card estimates are noisier (the variance between “agent nailed it” and “spec was wrong, full rewrite” is huge). Always report a range:

- **P50 estimate:** median throughput → 50% chance of finishing by this date
- **P85 estimate:** 0.7× median → 85% chance (use this for external commitments)
- **P15 estimate:** 1.4× median → optimistic case

A typical leadership update reads:

```
“We have 96 VH remaining. Rolling throughput is 24 VH/tick. P50 is 4 ticks (2.4 weeks), P85 is 6 ticks (3.6 weeks). Recommend communicating the P85.”
```

Burnup, not burndown

Classic burndown charts hide scope creep — a flat line could mean “we did nothing” or “we did 5 cards while 5 new ones were added.” The 2026 advanced agile metrics guidance is direct on this: “Burndown charts have a fatal flaw: they hide scope creep. If your team completes 5 points but the Product Owner adds 5 new points, the Burndown line looks flat. It looks like the team did nothing” [11]. That ambiguity is fatal in AAS, because **scope creep is now the dominant cause of slippage** (since coding isn’t).

Use a **burnup chart** with two lines per project:

- **Total scope (VH)** — trends upward when new cards are added; this is your scope-creep visibility
- **Completed VH** — trends upward as cards finish

The gap between them at any point is “VH remaining.” When the scope line jumps, you can have an explicit conversation with stakeholders about what was added and why — instead of it disappearing into a “the team

got slower” misread. As the same source notes: “If the scope line jumps up, you know exactly why the project is delayed. It wasn’t ‘slow developers’ — it was a changing target” [11].

Add a third line — **forecast completion line** — projecting completed VH forward at current throughput. Where it crosses the scope line is your projected done date.

Risk tiers: what humans actually review

Reviewing every AI PR is impossible at agent throughput. AAS uses three tiers, set on each Spec Card:

Tier	Examples	Review path
T1 — Autonomous	Tests, docs, type fixes, dependency bumps, isolated refactors	Agent + AI reviewer agent; human spot-checks 10% sample weekly
T2 — Reviewed	Standard features, bug fixes, schema changes (additive)	One human reviewer; AI pre-review surfaces concerns
T3 — Architect	Public APIs, security boundaries, schema migrations, performance-critical paths, cross-service contracts	Senior engineer + architect; mandatory design doc; staged rollout

This is the modern equivalent of “code review policy,” but explicit and tier-based. Superwise describes the same pattern as a “trust threshold” approach where you “loop in human reviewers only when trust thresholds are crossed” [12]. The Anthropic 2026 Agentic Coding Trends report confirms this is becoming standard: engineers tend to “delegate tasks that are easily verifiable — where they can relatively easily sniff-check on correctness,” while keeping conceptually difficult work for collaborative human-AI sessions [13].

The Reviewer Agent layer is well-supported empirically. The Qodo 2025 AI Code Quality report found that “usage of AI code reviews increased quality improvements to 81% (from 55%),” and an Atlassian RovoDev 2026 study showed “38.7% of comments left by AI agents in code reviews lead to additional code fixes” [14]. A longitudinal arXiv study of enterprise deployments found “an overall 31.8% reduction in PR review cycle time” with AI-assisted review [15].

The default for new card types should be T2 until you have data showing AI handles them reliably, at which point you can demote to T1.

Roles, redefined

Classic Scrum	AAS equivalent	What changes
Product Owner	Intent Owner	Defines acceptance tests, not just user stories. Specs are now executable artefacts.
Scrum Master	Flow Steward	Watches AES, HT, rework rate. Optimises human-agent handoffs, not standup attendance.
Developer	Agent Orchestrator	Spends 60-70% of time on specs and reviews, 20% on hard implementation work AI can't do, 10-20% on system design.
(new)	Architect / Tier-3 Reviewer	Owens the high-risk review path. Often a senior dev wearing a second hat.
(new)	Reviewer Agent	A second AI agent that reviews the first agent's output before a human sees it.

The “Agent Orchestrator” role description aligns with Scrum.org’s framing: “Human Developers must elevate their skill sets. They are no longer just writing boilerplate code; they act as ‘Agent Orchestrator’, tasked with validating the logic generated by their AI counterparts” [5]. Waydev’s analysis puts it more bluntly: “the old job: write code, review code, deploy code. The new job: orchestrate AI agents, steer parallel workstreams, make judgment calls AI can’t make, review AI output for strategic alignment” [16].

What stays the same from Scrum

Don’t throw out what works:

- **Empirical, iterative, inspect-and-adapt** — the philosophy is unchanged
- **A single prioritised backlog** — now of Spec Cards, ordered by Intent Owner
- **A regular cadence** — just shorter (3 days vs 2 weeks)
- **A retrospective** — just AI-summarised and quicker (15 min)
- **Working software at the end of every cycle** — actually easier to honour now
- **Definition of Done** — extended to include “validated by human reviewer at appropriate tier”

It’s also worth noting that Forrester reports “95% [of organisations] still find Agile relevant” [3], so this is a refinement of Scrum, not a rejection of it.

What dies from Scrum

- **Story points** — replaced by VH
- **Velocity-as-output-metric** — replaced by Tick Throughput + AES + Rework Rate (a vector, not a scalar)

- **2-week sprints** — replaced by 3-day ticks
 - **Daily synchronous standups** — replaced by AI-summarised async updates
 - **Burndown charts** — replaced by burnups (scope-creep visibility is non-negotiable)
 - **Manual sprint planning poker** — replaced by AI-assisted estimation that learns from your VH P/A history
-

The 30-day adoption plan

Week 1 — Instrument. Keep doing whatever you do today, but start logging: estimated vs actual hours per ticket, time from “AI-stuck” to “human-resumed,” and rework rate on AI PRs. You need a baseline.

Week 2 — Switch the unit. Convert your next sprint’s stories into Spec Cards with the six fields. Estimate in VH. Set risk tiers. Keep your old sprint length for now.

Week 3 — Shorten the cadence. Run two 3-day ticks in place of one sprint. Ditch daily standups for an AI-summarised async update. Hold a tick retro.

Week 4 — Switch the dashboard. Replace your burndown with a burnup. Start tracking the five metrics. Run forecasts using P50/P85 bands. Compare predicted vs actual — that’s your calibration signal.

After 30 days you’ll have your own throughput baseline, your own AES per workflow, and enough VH P/A data to forecast credibly. From there it’s normal continuous improvement.

Answering the management questions

The original brief was: *can it answer the same questions Scrum can?* Here’s the mapping:

Question	Scrum answer	AAS answer
How long will this take?	Story points ÷ velocity	VH ÷ tick throughput, with P50/P85 bands
Are we on schedule?	Burndown chart slope	Burnup forecast line vs scope line
What's next?	Top of sprint backlog	Top of tick backlog (cards at “ready”)
Are we getting better?	Velocity trend	Tick Throughput trend + AES trend + Rework Rate trend
Where's the risk?	Burndown flatlines, blockers in standup	Handoff Time spikes, AES drops, scope-line jumps
Are we delivering value?	Story acceptance rate	DORA metrics + escaped defects + business outcomes (unchanged from modern Scrum)
Who's working on what?	Sprint board columns	Tick board columns (Spec → Building → Review → Integrating → Done)

Everything Scrum could answer, AAS can answer — and several things Scrum couldn't measure (agent efficiency, handoff cost, true scope creep) become first-class.

One final caveat

This framework assumes a team that has crossed the threshold where AI handles >50% of code generation. Below that, classic Scrum with AI assistance (the “augmented Scrum” pattern) is probably enough. Don't over-engineer your process if your bottleneck is still typing.

The honest meta-point: **the field is unsettled**. Run AAS as an experiment for 4-6 ticks, measure aggressively, and adapt. The framework above is a starting point, not a destination.

Sources

- [1] Scrum.org. *From Velocity to “Agent Efficiency”: Evidence-Based Management for the AI Era*.
- [2] Agarwal, A. *New Rules for Estimating Software Development Time in AI-era*. Medium, May 2025.
- [3] InfoQ. *Does AI Make the Agile Manifesto Obsolete?* February 2026.
- [4] Beck, K. *Augmented Coding: Beyond the Vibes*. Tidy First Substack, 2025. See also SoftwareSeni's [analysis](#).
- [5] Scrum.org. *AI Augmented Scrum Framework: When Half Your Team is Autonomous Agents*. March 2026.
- [6] HPE Developer Portal. *Agentic AI in agile: Smarter sprints, faster retros*.
- [7] Augment Code. *Autonomous Development Metrics: KPIs That Matter for AI-Assisted Engineering Teams*. September 2025.

- [8] Scrum.org. *Managing Technical Debt in AI-Augmented Scrum Teams*.
- [9] LinearB. *AI Metrics: How to Measure Gen AI Code*.
- [10] Axify. *20 AI Performance Metrics to Follow in Software Development*. February 2026.
- [11] ScrumDay India. *Beyond Velocity: The 2026 Guide to Agile Metrics That Actually Matter*. January 2026.
- [12] Superwise. *Agile Is Dead: Long Live Agentic Development*. May 2025.
- [13] Anthropic. *2026 Agentic Coding Trends Report*.
- [14] Microsoft Community Hub. *An AI led SDLC: Building an End-to-End Agentic Software Development Lifecycle with Azure and GitHub*. February 2026.
- [15] *Intuition to Evidence: Measuring AI's True Impact on Developer Productivity*. arXiv preprint, September 2025.
- [16] Waydev. *The Engineering Leader's Paradox: When AI Writes the Code, What Do We Measure?* February 2026.

Further reading not directly cited

- Luelling, B. *SDLC for Agentic AI Engineering*. Medium, March 2026.
- *Agentsway — Software Development Methodology for AI Agents-based Teams*. arXiv, October 2025.
- IBM. *AI in the SDLC*.
- PwC Middle East. *Agentic SDLC in practice: the rise of autonomous software delivery 2026*. January 2026.
- GitHub. *spec-kit — Spec-Driven Development toolkit*.
- Beck, K. *Personal site — augmented coding research*.